ARCHITECTURE AND DECISION RECORD

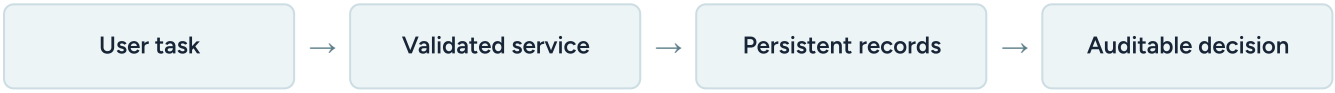
A service booking platform with transactional reservations

Rafael Dela Cruz · Web systems architect and database reviewer

Laravel | React | MySQL | Transactions

Problem and contribution

This original design case study explores how to coordinate scheduled appointments and walk-in service without allowing the same capacity to be allocated twice. I authored the requirements breakdown, architecture boundaries, data relationships, failure scenarios, and review rationale presented here. The design is a fictional presentation example, with no claim of a production deployment or measured client result.



Architecture decisions and tradeoffs

- Use a database transaction and row-level lock when reserving the final place in a slot; an interface-side availability check is insufficient.
- Separate appointment reservation from queue position: a reserved time does not imply that a student is present.
- Publish an explicit ratio or priority rule for appointments and walk-ins; do not let a clerk silently reorder the queue.

Core records

Record	Responsibility
users	One account can hold multiple appointments; staff access is restricted to assigned offices.
offices / services	Each service belongs to one office and defines an expected service duration.
appointment_slots	A slot has many appointments; a transactional count protects remaining capacity.
appointments	An appointment belongs to a student and one slot; duplicate active reservations are constrained.

The service layer owns authorization and business constraints. Interface validation improves the experience but cannot replace server checks. Event records preserve attributable decisions and their timing without logging unnecessary private content.

VERIFICATION AND REVIEW PRACTICE

Failure cases, evidence, and limitations

Proposed verification matrix

Scenario	Method	Expected behavior
Capacity boundary	Submit two simultaneous requests for the last slot.	One succeeds and one receives a clear capacity message; no overbooking.
Lifecycle integrity	Cancel, miss, recall, and complete tickets in valid and invalid orders.	Only allowed transitions persist and every transition records the acting staff member.
Fairness and recovery	Mix scheduled arrivals, walk-ins, late arrivals, and a restarted counter.	The published selection rule is preserved and no ticket is silently lost.
Usability comparison	Ask volunteers to book and locate their queue position using both workflows.	Record task success, time, errors, and participant comments without inventing improvements.

Technical review

Lock capacity rows during booking and use a unique request key to make retries idempotent. Test the last available place with concurrent requests rather than relying on a disabled button.

How to evaluate the design

Report booking conflict rate, task success, median and interquartile waiting time, and the difference between displayed and actual waits. Waiting begins at confirmed check-in, not at remote booking. Separate service types because a certificate request and a consultation may take different amounts of time.

The matrix specifies intended tests, not completed test results. A reviewer should capture the environment, initial records, inputs, expected state, observed state, and any unresolved defect before interpreting the outcome. Repeated runs should use separate unique request keys where relevant and preserve failures for analysis.

Scope limitations

No automatic staffing decisions, payment collection, SMS purchasing, biometric attendance, or guarantee that all queues become shorter.

The most important review question is whether the reservation and queue models remain consistent when many students book, cancel, or check in at the same time.

> **Authorship and reuse**
>
> Prepared by Rafael Dela Cruz as an original presentation work sample. All narrative, schemas, and test scenarios are original illustrative material. There are no external client documents, employment certificates, real personal records, or fabricated citations in this case study.